

Material de apoio ao módulo 2 - metrologia por imagem: automação com systemd

Prof. Felipe Walter Dafico Pfrimer

1 Objetivo do Módulo

No módulo anterior, vimos como acessar um sistema Linux por SSH, navegar pelo terminal, manipular arquivos e criar scripts Bash simples. Neste módulo, vamos usar esses conhecimentos para automatizar a execução de tarefas usando o `systemd`.

O objetivo principal é compreender como transformar uma rotina manual de captura de imagens em um processo controlado pelo sistema operacional. Para isso, vamos estudar três elementos:

Elementos da Automação

Elemento	Função	Exemplo no Módulo
script	Define o que será feito	Capturar uma imagem com <code>fswebcam</code>
service	Define como a tarefa será executada pelo sistema	Executar <code>captura.sh</code>
timer	Define quando a tarefa será executada	Repetir a captura a cada intervalo

Ao final da aula, o aluno deverá ser capaz de criar um script de captura, associá-lo a um serviço do `systemd`, repetir sua execução com um temporizador e consultar o estado e os logs da automação.

2 Por que Automatizar a Captura de Imagens?

Em um experimento de metrologia por imagem, a forma como os dados são coletados influencia diretamente a confiabilidade dos resultados. Se uma pessoa precisa lembrar de executar um comando a cada intervalo de tempo, podem ocorrer atrasos, esquecimentos ou variações no procedimento.

A automação reduz essa dependência da intervenção manual. Quando o sistema operacional executa a tarefa automaticamente, a rotina tende a ser mais regular, mais fácil de verificar e mais adequada para experimentos longos.

Benefícios da Automação

Aspecto	Problema Manual	Com Automação
Tempo	O operador pode atrasar ou esquecer	O sistema executa no intervalo definido
Repetibilidade	Cada execução pode variar	A mesma rotina é repetida
Registro	Erros podem passar despercebidos	O serviço pode gerar logs
Escala	Coletas longas cansam o operador	O sistema pode operar por horas ou dias

Automatizar não significa apenas “fazer o computador repetir uma tarefa”. Significa transformar uma rotina experimental em um procedimento mais controlado, verificável e reproduzível.

3 O que é systemd

O systemd é um sistema de inicialização e gerenciamento de serviços usado por muitas distribuições Linux. Ele é responsável por iniciar, controlar e monitorar processos importantes do sistema, como rede, login, SSH, sincronização de horário e rotinas definidas pelo administrador.

Na prática, o systemd permite que uma tarefa seja gerenciada pelo próprio sistema operacional, em vez de depender de um terminal aberto pelo usuário.

3.1 Unidades do systemd

O systemd organiza suas configurações em arquivos chamados unidades. Cada unidade descreve uma tarefa, um recurso ou um grupo de ações.

Tipos Comuns de Unidade

Tipo	Função	Exemplo
<code>.service</code>	Define um serviço ou tarefa executável	<code>captura.service</code>
<code>.timer</code>	Agenda a execução de um serviço	<code>captura.timer</code>
<code>.target</code>	Agrupa estados ou etapas do sistema	<code>multi-user.target</code>

Neste módulo, vamos trabalhar principalmente com arquivos `.service` e `.timer`.

4 O comando systemctl

O comando `systemctl` é a principal forma de controlar unidades do systemd. Com ele, podemos iniciar serviços, parar serviços, verificar estados, habilitar execução automática e listar temporizadores.

Comandos Básicos com `systemctl`

Comando	Função	Exemplo de Uso
<code>start</code>	Iniciar uma unidade	<code>sudo systemctl start captura.service</code>
<code>stop</code>	Parar uma unidade	<code>sudo systemctl stop captura.timer</code>
<code>status</code>	Verificar o estado	<code>systemctl status captura.service</code>
<code>enable</code>	Habilitar inicialização automática	<code>sudo systemctl enable captura.timer</code>
<code>disable</code>	Desabilitar inicialização automática	<code>sudo systemctl disable captura.timer</code>
<code>daemon-reload</code>	Recarregar arquivos modificados	<code>sudo systemctl daemon-reload</code>
<code>list-timers</code>	Listar temporizadores ativos	<code>systemctl list-timers</code>

Observação: sempre que um arquivo `.service` ou `.timer` for criado ou modificado, é necessário executar `sudo systemctl daemon-reload`. Esse comando informa ao `systemd` que ele deve recarregar suas configurações.

5 Teste Manual Antes da Automação

Antes de criar um serviço ou um timer, é essencial testar a captura manualmente. Se a câmera, o comando ou o script não funcionam no terminal, também não funcionarão corretamente quando forem chamados pelo `systemd`.

Verificação Manual da Câmera

Comando	Função	Exemplo de Uso
<code>ls /dev/video*</code>	Verificar dispositivos de vídeo	<code>ls /dev/video*</code>
<code>type fswebcam</code>	Verificar se <code>fswebcam</code> existe	<code>type fswebcam</code>
<code>mkdir -p</code>	Criar pasta para imagens	<code>mkdir -p imagens</code>
<code>fswebcam</code>	Capturar imagem da webcam	<code>fswebcam -r 1280x720 imagens/teste.jpg</code>
<code>ls -lh</code>	Conferir o arquivo gerado	<code>ls -lh imagens</code>

Uma regra prática importante é:

Se não funciona manualmente, ainda não está pronto para virar automação.

6 Script de Captura Simples

O primeiro modelo de automação usa um script que faz apenas uma captura e termina. Esse formato é adequado para ser chamado por um timer, pois cada disparo executa uma nova coleta.

6.1 Exemplo de Script

O arquivo pode ser chamado de `captura.sh`. O exemplo abaixo salva a imagem em uma pasta chamada `imagens`, usando data e hora no nome do arquivo.

Script `captura.sh`

Linha	Código
-------	--------

1	<code>#!/bin/bash</code>
2	
3	<code>mkdir -p /home/g0/imagens</code>
4	
5	<code>ts=\$(date +%Y%m%d_%H%M%S)</code>
6	<code>arquivo="/home/g0/imagens/foto_\${ts}.jpg"</code>
7	
8	<code>echo "Capturando imagem em \$arquivo"</code>
9	<code>/usr/bin/fswebcam -r 1280x720 "\$arquivo"</code>
10	<code>echo "Captura finalizada"</code>

O usuário `g0` deve ser substituído pelo usuário correto de cada grupo, como `g1`, `g2`, `g3` ou `g4`. Em arquivos usados pelo `systemd`, é recomendável usar caminhos absolutos, como `/home/g0/captura.sh`, em vez de caminhos relativos, como `./captura.sh`.

6.2 Executando o Script Manualmente

Teste Manual do Script

Comando	Função	Exemplo de Uso
<code>chmod +x</code>	Tornar o script executável	<code>chmod +x captura.sh</code>
<code>./captura.sh</code>	Executar a captura manualmente	<code>./captura.sh</code>
<code>ls -lh</code>	Verificar imagens salvas	<code>ls -lh imagens</code>

Esse teste confirma se o script consegue criar a pasta, gerar o nome do arquivo e chamar a câmera corretamente.

7 Serviço para Captura Simples

Um serviço do `systemd` descreve como uma tarefa deve ser executada. Neste exemplo, usaremos um serviço do tipo `oneshot`, pois a tarefa executa uma vez e termina.

7.1 Estrutura Geral de um Serviço

Seções de um Arquivo `.service`

Seção	Função	Exemplo
[Unit]	Descreve a unidade e dependências	Description=Captura de imagem
[Service]	Define como executar a tarefa	ExecStart=...
[Install]	Define como habilitar a unidade	WantedBy=multi-user.target

7.2 Exemplo de `captura.service`

Arquivo `/etc/systemd/system/captura.service`

Linha	Código
1	[Unit]
2	Description=Captura de imagem com script do usuario g0
3	After=network.target
4	
5	[Service]
6	Type=oneshot
7	User=g0
8	WorkingDirectory=/home/g0
9	ExecStart=/home/g0/captura.sh
10	
11	[Install]
12	WantedBy=multi-user.target

Campos Importantes do Serviço

Campo	Função	Neste Exemplo
Type=oneshot	Executa uma tarefa e termina	Uma captura por execução
User=g0	Define o usuário da execução	Usa o diretório do grupo g0
WorkingDirectory	Define o diretório de trabalho	/home/g0
ExecStart	Define o comando executado	/home/g0/captura.sh

7.3 Testando o Serviço

Controle do Serviço

Comando	Função	Exemplo de Uso
<code>daemon-reload</code>	Recarregar unidades modificadas	<code>sudo systemctl daemon-reload</code>
<code>start</code>	Executar o serviço	<code>sudo systemctl start captura.service</code>
<code>status</code>	Verificar o estado	<code>systemctl status captura.service</code>
<code>journalctl</code>	Ver mensagens registradas	<code>journalctl -u captura.service -n 20</code>

Depois de iniciar o serviço, a pasta de imagens deve conter uma nova captura. Se isso não acontecer, o primeiro passo é verificar o estado com `systemctl status` e depois consultar os logs com `journalctl`.

8 Timer para Repetir a Captura

Um timer agenda a execução de um serviço. Ele não executa a captura diretamente; ele apenas dispara o serviço no intervalo definido.

O serviço define o que fazer. O timer define quando fazer.

8.1 Estrutura Geral de um Timer

Seções de um Arquivo `.timer`

Seção	Função	Exemplo
<code>[Unit]</code>	Descreve o temporizador	<code>Description=Timer de captura</code>
<code>[Timer]</code>	Define o agendamento	<code>OnUnitActiveSec=1min</code>
<code>[Install]</code>	Define como habilitar o timer	<code>WantedBy=timers.target</code>

8.2 Exemplo de `captura.timer`

Arquivo `/etc/systemd/system/captura.timer`

Linha	Código
1	[Unit]
2	Description=Timer para executar captura de imagem periodicamente
3	
4	[Timer]
5	OnBootSec=30s
6	OnUnitActiveSec=1min
7	Unit=captura.service
8	
9	[Install]
10	WantedBy=timers.target

Neste exemplo, o timer aguarda 30 segundos após a inicialização e, depois, executa o serviço novamente a cada 1 minuto. O intervalo de 1 minuto é útil em aula porque permite observar as imagens sendo geradas. Em uma aplicação real, o intervalo pode ser maior.

8.3 Controlando o Timer

Comandos para o Timer

Comando	Função	Exemplo de Uso
<code>enable --now</code>	Habilitar e iniciar imediatamente	<code>sudo systemctl enable --now captura.timer</code>
<code>status</code>	Verificar estado do timer	<code>systemctl status captura.timer</code>
<code>list-timers</code>	Listar temporizadores ativos	<code>systemctl list-timers</code>
<code>stop</code>	Parar temporariamente	<code>sudo systemctl stop captura.timer</code>
<code>disable</code>	Desabilitar inicialização automática	<code>sudo systemctl disable captura.timer</code>

Quando o timer está ativo, o serviço será chamado automaticamente. Para verificar se as capturas ocorreram, consulte a pasta de imagens e os logs do serviço:

Verificação da Captura Agendada

Comando	Função	Exemplo de Uso
<code>ls -lh</code>	Verificar arquivos gerados	<code>ls -lh imagens</code>
<code>journalctl</code>	Ver logs da execução	<code>journalctl -u captura.service -n 30</code>

9 Logs e Diagnóstico

Em tarefas automáticas, os erros nem sempre aparecem na tela. Por isso, os logs são uma parte essencial do trabalho com `systemd`.

Diagnóstico de Problemas

Pergunta	Comando Útil	O que Observar
O serviço executou?	<code>systemctl status captura.service</code>	Estado, código de saída e mensagens
O timer está ativo?	<code>systemctl list-timers</code>	Próxima execução e última execução
Houve erro no script?	<code>journalctl -u captura.service -n 30</code>	Mensagens do script e falhas
As imagens foram geradas?	<code>ls -lh imagens</code>	Arquivos com data e hora

Problemas comuns incluem caminho incorreto, falta de permissão de execução, usuário errado no serviço, câmera indisponível ou ausência do comando `fswebcam`.

10 Coleta Contínua com Apenas um Serviço

O modelo com `service + timer` é adequado quando uma tarefa pode ser executada uma vez e terminar. Porém, para coletas muito frequentes, pode ser interessante usar um script contínuo, que fica rodando em loop.

Nesse segundo modelo, não usamos timer. O próprio script controla o intervalo entre capturas usando `sleep`.

10.1 Comparação entre os Modelos

Timer versus Serviço Contínuo

Modelo	Como Funciona	Quando Usar
<code>service + timer</code>	O timer chama o serviço repetidamente	Intervalos moderados, como minutos
<code>service contínuo</code>	O serviço inicia um script que permanece em execução	Intervalos curtos, como segundos

10.2 Script de Captura Contínua

O script abaixo captura imagens continuamente e aguarda 15 segundos entre uma captura e outra.

Script `captura_continua.sh`

Linha	Código
-------	--------

1	<code>#!/bin/bash</code>
2	
3	<code>mkdir -p /home/g0/imagens_continuas</code>
4	
5	<code>while true; do</code>
6	<code> ts=\$(date +%Y%m%d_%H%M%S)</code>
7	<code> arquivo="/home/g0/imagens_continuas/foto-\${ts}.jpg"</code>
8	
9	<code> echo "Capturando imagem em \$arquivo"</code>
10	<code> /usr/bin/fswebcam -r 1280x720 "\$arquivo"</code>
11	
12	<code> sleep 15</code>
13	<code>done</code>

Como esse script não termina sozinho, o teste manual precisa ser interrompido com `Ctrl+C`.

10.3 Serviço Contínuo

Para um script contínuo, usamos um serviço do tipo `simple`. O processo iniciado pelo serviço permanece ativo enquanto o script estiver rodando.

Arquivo `/etc/systemd/system/captura-continua.service`

Linha	Código
-------	--------

1	<code>[Unit]</code>
2	<code>Description=Captura continua de imagens com fswebcam</code>
3	<code>After=network.target</code>
4	
5	<code>[Service]</code>
6	<code>Type=simple</code>
7	<code>User=g0</code>
8	<code>WorkingDirectory=/home/g0</code>
9	<code>ExecStart=/home/g0/captura_continua.sh</code>
10	<code>Restart=always</code>
11	
12	<code>[Install]</code>
13	<code>WantedBy=multi-user.target</code>

Controle do Serviço Contínuo

Comando	Função	Exemplo de Uso
start	Iniciar a coleta contínua	<code>sudo systemctl start captura-continua.service</code>
status	Verificar se está rodando	<code>systemctl status captura-continua.service</code>
journalctl	Ver mensagens da coleta	<code>journalctl -u captura-continua.service -n 30</code>
stop	Parar a coleta contínua	<code>sudo systemctl stop captura-continua.service</code>

11 Cuidados Importantes

Ao trabalhar com `systemd`, pequenos detalhes podem impedir a automação de funcionar. A tabela abaixo resume cuidados úteis durante a prática.

Checklist de Automação

Cuidado	Motivo	Como Conferir
Testar manualmente	Isola problemas do script	<code>./captura.sh</code>
Usar caminho absoluto	O serviço não depende do terminal	<code>/home/g0/captura.sh</code>
Dar permissão de execução	O script precisa poder rodar	<code>chmod +x captura.sh</code>
Recarregar o <code>systemd</code>	Arquivos novos precisam ser lidos	<code>sudo systemctl daemon-reload</code>
Verificar logs	Erros podem não aparecer na tela	<code>journalctl -u captura.service</code>
Parar serviços contínuos	Evita capturas indefinidas	<code>systemctl stop ...</code>

Antes de pedir ajuda, é recomendável verificar três pontos: se o script roda manualmente, se o serviço apresenta erro no `status` e se o `journalctl` mostra alguma mensagem relacionada a caminho, permissão ou câmera.

12 Conclusão

Neste módulo, vimos como o `systemd` pode ser usado para automatizar rotinas de captura de imagens em um sistema Linux. A partir de um script Bash, criamos um serviço para executar uma tarefa e um timer para repetir essa tarefa em intervalos definidos.

Também vimos uma alternativa para coletas rápidas: um serviço contínuo que inicia um script com `loop` e `sleep`. A diferença entre os modelos é fundamental: no primeiro, o timer controla o intervalo; no segundo, o intervalo está dentro do próprio script.

Em metrologia por imagem, essa automação ajuda a reduzir erros manuais, organizar dados e melhorar a repetibilidade da coleta. Com serviços, timers e logs, o experimento deixa de

depende apenas da ação direta do operador e passa a ser controlado por uma rotina configurada no sistema.